

Práctica 2 - Lógica Digital Secuencial - Parte A

Sistemas Digitales

Segundo Cuatrimestre 2026

Circuitos Secuenciales

Ejercicio 1 Considere una compuerta NOT cuya salida **q** se conecta directamente a su entrada. La compuerta tiene un retardo de propagación de 5 ns: cuando cambia su entrada, su salida toma el valor negado 5 ns después. Inicialmente, **q** = 0.

- Complete el valor de **q** desde 0 ns hasta 30 ns, indicando cada instante en el que cambia.
- ¿Existe un valor binario estable para **q** que satisfaga la conexión?
- Repita el análisis para un lazo formado por dos compuertas NOT en serie. Las señales son **q0** y **q1**, cada compuerta tarda 5 ns y al comienzo **q0** = 0 y **q1** = 0.
- Determine los estados estables del circuito de dos inversores. ¿Alcanza con conocer su conexión para predecir cuál de ellos se alcanzará?
- Explique por qué un lazo combinacional accidental no es una forma apropiada de almacenar información en un diseño sincrónico.

Ejercicio 2 Un latch SR se construye con dos compuertas NOR realimentadas y se define por las ecuaciones:

$$\begin{aligned} Q_{\text{next}} &= \sim (R \mid Q_{\text{bar}}) \\ Q_{\text{bar_next}} &= \sim (S \mid Q) \end{aligned}$$

Las entradas **S** y **R** son activas en alto. Cuando el circuito está en un estado válido, **Qbar** es el complemento de **Q**.

- Para cada combinación de **S** y **R**, determine los valores estables de **Q** y **Qbar**. Clasifique cada caso como **mantener**, **set**, **reset** o **inválido**.
- Partiendo de **Q** = 0, analice la secuencia siguiente. Cada fila se mantiene el tiempo suficiente para estabilizar el circuito.

Paso	S	R	Q resultante
0	0	0	
1	1	0	
2	0	0	
3	0	1	
4	0	0	

- ¿Por qué la fila **S** = 1, **R** = 1 no debe usarse para almacenar un dato? Analice además qué puede ocurrir al volver simultáneamente a **S** = 0, **R** = 0.
- Escriba una tabla característica en términos de **S**, **R**, **Q** y **Q_next**.

Ejercicio 3 Compare dos componentes inicialmente en **Q = 0**:

- un latch D transparente cuando **enable = 1**;
- un flip-flop D que captura el dato en cada flanco ascendente de **clk**.

En el latch, use **enable = clk**. Las señales cambian según la tabla. Un guion indica que la señal conserva su valor anterior.

Instante	clk	D	Evento
0 ns	0	0	inicio
5 ns	1	-	flanco ascendente
7 ns	-	1	cambia D
10 ns	0	-	flanco descendente
12 ns	-	0	cambia D
15 ns	1	-	flanco ascendente
17 ns	-	1	cambia D
20 ns	0	-	flanco descendente

- a) Dibuje **Q_latch** y **Q_ff** entre 0 ns y 20 ns.
- b) Indique en qué intervalos el latch es transparente.
- c) Indique qué valor captura el flip-flop en cada flanco ascendente.
- d) Explique con sus palabras la diferencia entre sensibilidad por nivel y por flanco.
- e) Implemente ambos componentes en SystemVerilog y compruebe la traza con un test-bench. No use un latch como elemento de estado en los ejercicios sincrónicos siguientes.

Ejercicio 4 Un registro de 4 bits tiene entradas **clk**, **reset**, **write_enable** y **data_in[3:0]**, y salida **data_out[3:0]**. Su actualización en cada flanco ascendente es:

- a) si **reset = 1**, almacena **4'b0000**;
- b) en caso contrario, si **write_enable = 1**, almacena **data_in**;
- c) en otro caso, conserva su contenido.

Antes del primer flanco, el contenido es desconocido. Complete la salida luego de cada flanco:

Flanco ascendente	reset	write_enable	data_in	data_out luego del flanco
1	1	1	4'b1010	
2	0	0	4'b1111	
3	0	1	4'b0011	
4	0	0	4'b1001	
5	1	0	4'b0101	
6	0	1	4'b1100	

- a) Justifique el resultado de cada fila.

- b) ¿Qué ocurriría en el primer flanco si **reset** no estuviera activo?
- c) ¿Qué prioridad tiene **reset** respecto de **write_enable** y en qué fila puede observarse?
- d) Si el período del reloj es 10 ns, ¿cuál es su frecuencia? ¿Cuánto tiempo transcurre entre el primer y el sexto flanco ascendente?

Ejercicio 5 Diseñe el módulo **register** con parámetro **N**, inicialmente 8, y la siguiente interfaz:

```
module register #(
    parameter int N = 8
) (
    input  logic      clk,
    input  logic      reset,
    input  logic      write_enable,
    input  logic [N-1:0] data_in,
    output logic [N-1:0] data_out
);
```

Debe cumplir la prioridad y el comportamiento del ejercicio anterior.

- a) Escriba primero una tabla de actualización para **data_out_next**.
- b) Implemente el módulo usando **always_ff @(posedge clk)**.
- c) Construya un testbench autocontrolado que verifique reset, carga y conservación. Ante una diferencia debe ejecutar **\$error**.
- d) Ejecute el mismo testbench para **N = 4** y **N = 8**.
- e) Modifique deliberadamente la implementación para usar asignaciones bloqueantes dentro de **always_ff**. Compare el resultado del linter y explique por qué la convención requerida es usar asignaciones no bloqueantes.

Ejercicio 6 Diseñe un registro de 8 bits con entradas **clk**, **reset**, **load**, **shift**, **serial_in** y **data_in[7:0]**, y salida **data_out[7:0]**. En cada flanco ascendente debe aplicar esta prioridad:

- a) si **reset = 1**, almacenar **8'b0**;
- b) si no, si **load = 1**, almacenar **data_in**;
- c) si no, si **shift = 1**, desplazar el contenido una posición hacia el bit más significativo: el nuevo bit 0 toma **serial_in**, el nuevo bit 7 toma el antiguo bit 6;
- d) si ninguna condición se cumple, conservar el contenido.

Se pide:

- a) Escribir una expresión para cada bit del próximo estado durante un desplazamiento.
- b) Dibujar un datapath posible usando flip-flops y multiplexores.
- c) Completar el contenido luego de cada flanco para esta secuencia:
- d) Implementar y verificar el módulo en SystemVerilog. La operación debe modificar el estado almacenado, no solamente transformar la salida.

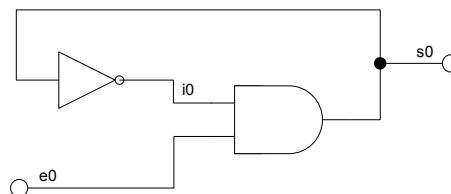
Flanco	reset	load	shift	serial_in	data_in	Contenido
1	1	0	0	0	8'h00	
2	0	1	0	0	8'h35	
3	0	0	1	1	8'h00	
4	0	0	1	0	8'h00	
5	0	0	0	1	8'hFF	
6	0	1	1	1	8'hA6	

Ejercicio 7 Un flip-flop D tiene tiempo de setup $t_{\text{setup}} = 2 \text{ ns}$, tiempo de hold $t_{\text{hold}} = 1 \text{ ns}$ y retardo clock-to-Q $t_{\text{cq}} = 1 \text{ ns}$. Se produce un flanco ascendente del reloj en 20 ns.

- Indique el intervalo alrededor del flanco durante el cual **D** debe permanecer estable.
- Determine cuáles de estos casos cumplen ambas restricciones:
 - A: **D** cambia en 17 ns y vuelve a cambiar en 22 ns.
 - B: **D** cambia en 19 ns y vuelve a cambiar en 23 ns.
 - C: **D** cambia en 16 ns y vuelve a cambiar en 20.5 ns.
- Si se cumplen las restricciones, ¿a partir de qué instante puede suponerse estable el nuevo valor de **Q**?
- Explique por qué una simulación funcional ideal puede no mostrar el problema producido por violar setup o hold.

Ejercicio 8

- Escribir el diagrama temporal para el siguiente circuito secuencial desde 0 ns hasta 65 ns, suponiendo
 - un retardo de 15 ns para la compuerta AND,
 - un retardo de 5 ns para la compuerta NOT,
 - en el tiempo 0 ns la señal e_0 cambia a 1, inicialmente en 0.
 - las señales i_0 y s_0 tienen valor 1 y 0 respectivamente en el tiempo 0 ns.
 - suponer que los componentes empiezan a estabilizarse cuando sus señales de entrada están estables.



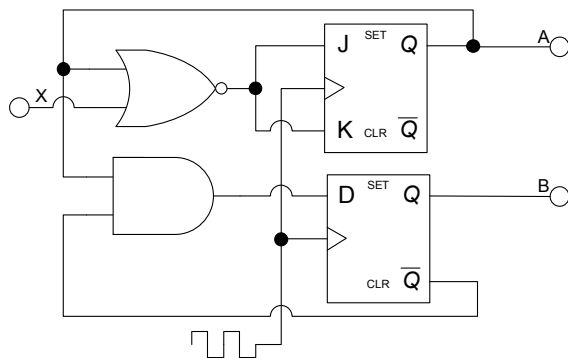
- ¿Podría alcanzar s_0 un valor estable en el punto anterior? ¿Y en el caso en que e_0 fuera 0 en lugar de 1, se estabilizaría?
- Complete la tabla de verdad para éste circuito, suponiendo que i_0 no está conectado a s_0 con el negador.

e_0	i_0	s_0
0	0	
0	1	
1	0	
1	1	

- d) Complete la tabla de verdad tomando el circuito retroalimentado, teniendo en cuenta que s_0 es el valor almacenado en el circuito, y que s'_0 es el nuevo valor que almacenará.

e_0	s_0	s'_0
0	0	
0	1	
1	0	
1	1	

Ejercicio 9 Completar la tabla característica para el siguiente circuito secuencial con los valores *estables* de la salida. Luego implementarlo en SystemVerilog.



A	B	X	A'	B'
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

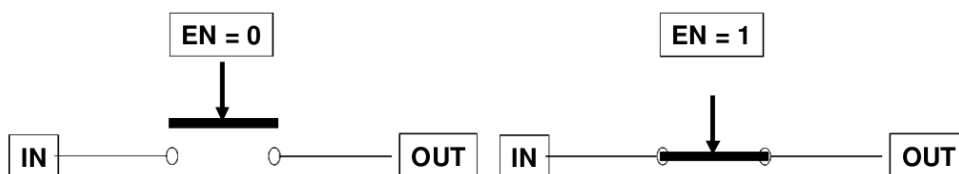
Ejercicio 10 Se desea construir un circuito que controle una señal de tránsito. Para esto se cuenta con un *clock* de 100 Hz. Los tiempos de delay de las compuertas pueden considerarse despreciables. Este circuito debe responder al siguiente comportamiento

salida	0 sg	1 sg	2 sg	3 sg	4 sg	k sg
s_0	1	0	0	0	1	...
s_1	0	1	0	0	0	...
s_2	0	0	1	0	0	...
s_3	0	0	0	1	0	...

Se pide entonces para conseguir dicho circuito:

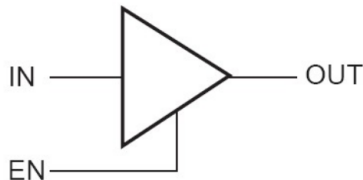
- Construir un registro incremental por señal de reloj de 7 bits.
- Construir un circuito combinatorio con 7 bits de entrada, que devuelva 1 cuando la señal sea 1100100.
- Construir un registro de rotación de 4 bits.
- Conectar los componentes anteriores de manera tal de conseguir el circuito pedido.

Ejercicio 11 Los buffers de tres estados son componentes electrónicos que se comportan como el siguiente esquema eléctrico:



EN: enable - IN: input - OUT: output

El mismo se representa como:



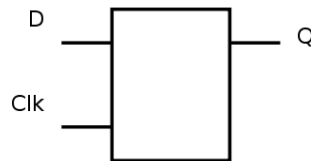
La tabla de verdad es:

IN	EN	OUT
0	1	0
1	1	1
x	0	Hi-Z

Hi-Z significa “alta impedancia”, es decir, que tiene una resistencia alta al pasaje de corriente. Como consecuencia de esto, podemos considerar al *pin* **OUT** como desconectado del circuito.

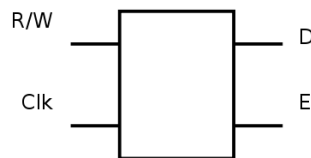
El biestable D (ver figura) se comporta de la siguiente manera:

- El componente emite por la señal Q el valor almacenado.
- Cuando la señal de reloj se activa, $Clk = 1$, el componente guarda el valor de la señal de entrada D .

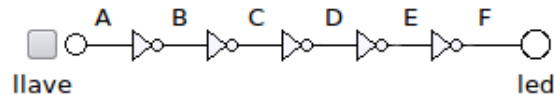


a) usando componentes lógicos comunes y de tres estados, se pide crear un componente *registro* con las señales D (data), $R/\overline{W}$ (read/write), Clk (clock) y E (enable), con el siguiente comportamiento:

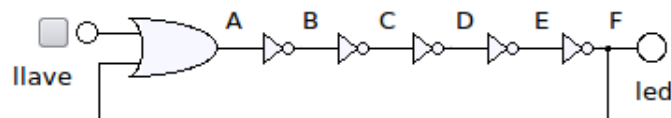
- Si $E = 1$, $Clk = 1$ y $R/\overline{W} = 1$, entonces el componente sensa el valor de D y lo almacena.
- Si $E = 1$, $Clk = 1$ y $R/\overline{W} = 0$, entonces el componente escribe el valor almacenado en D .
- Si $E = 0$ el componente pone la señal D en $Hi - Z$.



Ejercicio 12 Suponiendo que se cuenta con el siguiente circuito y que cada compuerta **Not** tiene un retardo de 5 ns y que la propagación por el medio físico es instantánea



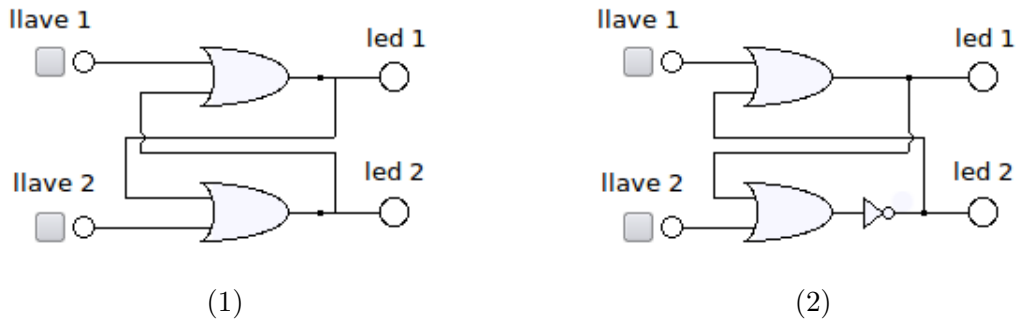
- dibuje un diagrama de tiempos que muestre la propagación de la señal desde que se presiona el botón hasta que se enciende el led.
- ¿qué ocurre si llave se desconecta luego de 15 ns ? ¿cuánto dura encendido el led?
- Si se modificara el circuito de la siguiente manera:



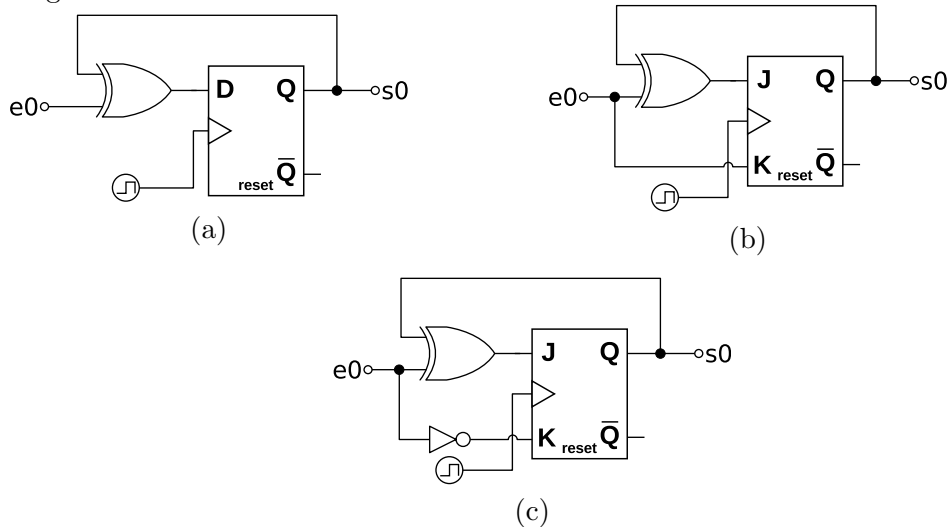
¿qué pasaría si la llave se conecta y se desconecta luego de 15 ns ? Compruebe su hipótesis construyendo un nuevo diagrama de tiempos, asumiendo que el retardo del Or también es de 5 ns y que inicialmente todos los conectores están propagando un valor lógico equivalente a cero.

- d) Si el diagrama anterior tuviera una cantidad par de compuertas Not, ¿se comportaría de la misma manera?

Ejercicio 13 Asumiendo los mismos tiempos de retardo que en el ejercicio anterior (la compuerta OR tarda lo mismo que la AND), describa el comportamiento de los siguientes circuitos si la *llave 1* se mantiene desactivada (en cuyo caso emite un 0) y la *llave 2* se acciona por 15 ns (emitiendo un 1) y luego se vuelve a desactivar. Asuma que, inicialmente, todas las líneas están en cero.



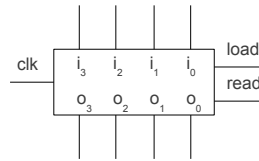
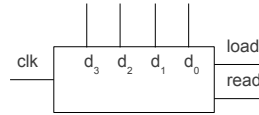
Ejercicio 14 Escriba tablas características que especifiquen el comportamiento de cada uno de los siguientes circuitos secuenciales:



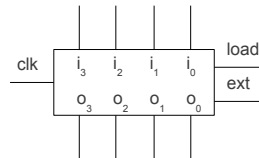
Ejercicio 15 Registro bidireccional Diseñar un registro *bidireccional* de cuatro bits. Este tipo de registros es un circuito con tres entradas (*load*, *read*, *clk*) y cuatro señales de entrada y salida (d_0 a d_3). Su funcionamiento es el siguiente: si la señal *load* vale 1 cuando *clk* alcanza su flanco ascendente, almacena los valores recibidos en d_0 a d_3 ; en cambio, si *read* está alta, se emite el valor almacenado en el registro por esas mismas líneas¹. Las señales *read* y *load* nunca valen 1 simultáneamente.

Ejercicio 16 Registro de salida restringida. Diseñar un registro *de salida restringida* de cuatro bits. Este tipo de registros es un circuito de siete entradas (i_0 a i_3 , *load*, *clk* y *read*) y cuatro salidas (o_0 a o_3), muy similar al registro simple (ejercicio ??) pero que sólo emite su salida por las líneas o_0 a o_3 si *read* está alta cuando *clk* alcanza su flanco ascendente. Dicha salida se debe mantener hasta el próximo flanco ascendente en *clk* donde *read* esté baja.

¹ Ayuda: utilice componentes de tres estados.

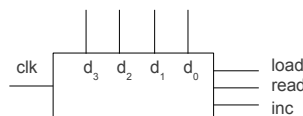


Ejercicio 17 Extensor de signo Diseñar un registro *extensor de signo* de dos a cuatro bits. Del mismo modo que un registro simple (ver ejercicio ??) este circuito toma el valor de sus cuatro entradas i_0 a i_3 cuando su señal clk atraviesa su flanco ascendente si la entrada $load$ vale 1. Por sus líneas de salida (o_0 a o_3) se emite el valor almacenado si la señal ext está baja; por el contrario, si vale 1, se emite una representación de cuatro *bits* del número almacenado en los dos *bits* menos significativos del registro, interpretados como un entero codificado en complemento a 2.



Ejercicio 18 Desplazador Diseñar un registro *desplazador a izquierda* de cuatro bits. Del mismo modo que un registro simple (ver ejercicio ??) este circuito toma el valor de sus cuatro entradas i_0 a i_3 cuando su señal clk atraviesa su flanco ascendente, si la entrada $load$ vale 1. Por sus líneas de salida (o_0 a o_3) se emite el valor almacenado si la señal shl está baja; por el contrario, si vale 1, se emite ese valor desplazado en uno hacia la izquierda.

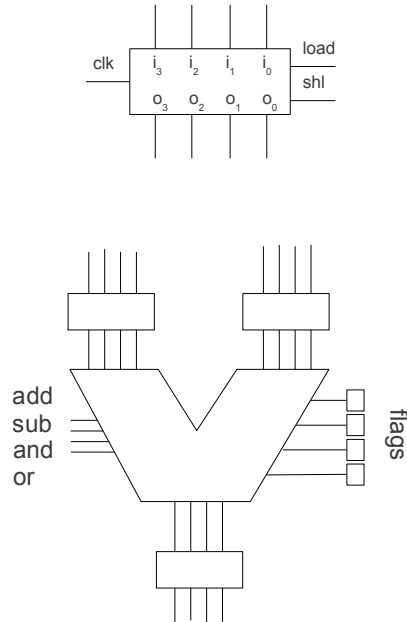
Ejercicio 19 Registro Auto-incrementador Diseñar un registro *bi-direccional auto-incrementador* de cuatro bits. Este circuito funciona de la misma manera que un registro bi-direccional (ver ejercicio 11), pero tiene una línea de entrada extra inc . Si, cuando clk alcanza su flanco ascendente, inc vale 1, el valor almacenado por el registro se incrementa en uno. Nuevamente, sólo una de las tres líneas de control ($load$, $read$ e inc) puede valer 1 cuando clk alcanza su flanco ascendente.



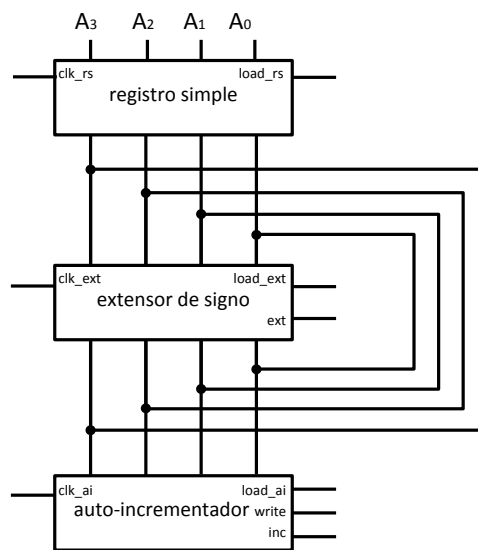
Ejercicio 20 ALU

Diseñe una ALU con las siguientes características:

- cuatro señales de entrada que indican la operación a realizar: **add**, **sub**, **and**, **or**;
- dos registros simples, donde se almacenan los operandos a utilizar;
- un registro de salida restringida, donde se almacena el resultado;
- cuatro *flags* cuyos valores son determinados por la última operación realizada.

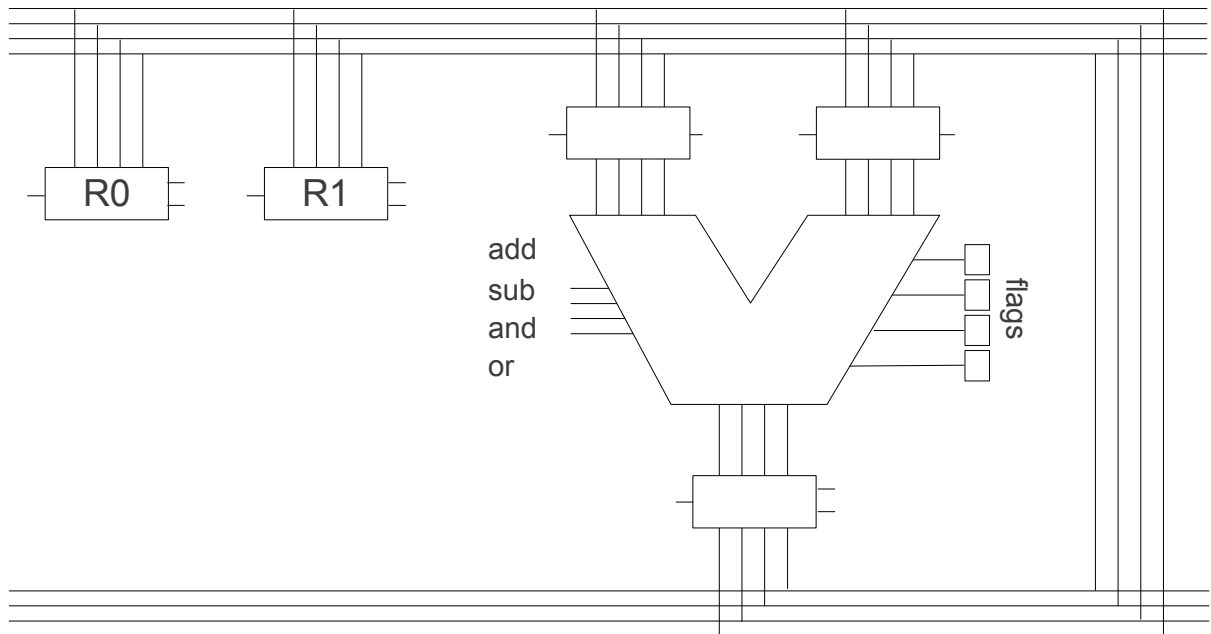


Ejercicio 21 Dado el siguiente circuito y asumiendo que el registro auto-incrementador comienza almacenando el valor 0000, indique mediante un diagrama de tiempos la secuencia de activaciones y desactivaciones de señales de control necesarias para que, en el registro simple, se almacenen las representaciones en complemento a 2 de los valores: 0, 2, -1, 0, 2, etc.



Ejercicio 22

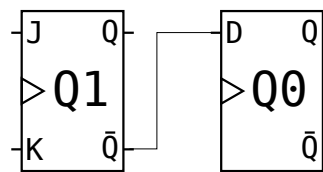
Dado el siguiente circuito, indique mediante un diagrama de tiempos la secuencia de activaciones y desactivaciones de señales de control necesarias para que el valor almacenado en el registro bidireccional (ejercicio 11) R0 se sume al valor del registro bidireccional R1 y el resultado se almacene en el registro R0.



Para seguir practicando

Ejercicio 23 a) Dada la siguiente tabla que se encontró incompleta y el circuito también incompleto, completarlos para que sean consistentes.

$Q_1(t)$	$Q_0(t)$	$Q_1(t+1)$	$Q_0(t+1)$
0	0	1	?
0	1	0	?
1	0	0	?
1	1	0	?



b) ¿El circuito anterior cambia su valor a cada clock? ¿O para alguna entrada es estable?